Busy .NET Developer's Guide to Orleans

Ted Neward

Neward & Associates

ted.neward@newardassociates.com
http://blogs.newardassociates.com

Objectives

Our goal here is to:

- Explore what a "virtual actor" is and how it works
- See how Orleans implements these virtual actors
- Talk about Orleans-centric distributed system design

In a nutshell

MICROSOFT ORLEANS: OVERVIEW

Microsoft Orleans: Overview

What is it?

- actors model for CLR technology stacks
  - "grains"
  - stateless or persistent
  - virtual actors: "always alive" semantics
- open-source
  - https://github.com/microsoft/orleans
- used on some sizable projects
  - Halo 4, Halo 5

What's important to know?

ORLEANS CONCEPTS

Orleans Concepts

Core concepts:

- Grain
- Silo
- Cluster
- Host

Orleans Concepts

Grain

- "Virtual Actor": Identity + Logic + State
- defined observable/"hook"able lifecycle
- Single-threaded, serial message receipt
- single living instance: "activation"

Orleans Concepts

Grain identity

- Objects have implicit identity (memory address)
- Actors use more generalized identity
- Orleans grains must have "long-lived" identity
- Orleans uses one of five options for identity:
  - long
  - GUID
  - string
  - GUID + string (compound)

Orleans Concepts

Grain state

- Grains can be entirely stateless
  - singletons, in essence
  - can allow multiple activations
- Grains can hold state
  - state is held across lifetime of server
  - state can be persisted to external storage

Orleans Concepts

Grain interface

- Describes the messaging surface
- uses CLR interfaces and async idioms
- only type visible to clients
- "IDL" for Orleans

Orleans Concepts

Message

- bundle of data passed across the wire
- binary format; serialized (deep copy)
- actual runtime type passed (not declared type)
- Orleans generates "serializers" as part of build
- can be invoked `await` (preserves order) or async (no ordering)

Orleans Concepts

Silo

- server instance/process, containing grains
- manage allocation/lifecycle of grains
  - provides location transparency
  - forces "remote-first" mentality
- designed to work together in a cluster
- defined observable/"hook"able lifecycle

Orleans Concepts

Cluster

- collection of silos
- responsible for grain placement in Silos
  - random
  - local-to-caller (server)
  - hash-based
  - activation-count-based
  - configurable and customizable

Orleans Concepts

Host

- process which contains the running server(s)
- most often a Console, Service, or ASP.NET app
- heavy use of Microsoft Hosting classes

Orleans Concepts

Rules of thumb

- Grains always exist
  - runtime activates them on demand
  - runtime deactivates them "later" or on request
- Everything is a Task or Task-of-T
  - enforcing/requiring asynchronicity

Zero to Go in....

ORLEANS GETTING STARTED

Orleans Getting Started

Platform requirements

- Microsoft .NET (.NET 10.0)
- ... and a network

Orleans Getting Started

Hello, Orleans

- https://github.com/tedneward/Demo-Orleans/HelloWorld
- four projects:
  - interfaces (IDL)
  - implementations
  - client
  - server/host
- any CLR language (most often C#)

Orleans Getting Started

Interfaces

- ClassLibrary
- requires
  - Microsoft.Orleans.Core.Abstractions
  - Microsoft.Orleans.CodeGenerator.MSBuild

Orleans Getting Started

IHelloWorld

```
public interface IHelloWorld : Orleans.IGrainWithIntegerKey
{
    Task<string> SayHello(string name);
}
```

Orleans Getting Started

Grains (implementations)

- ClassLibrary
- requires
  - Microsoft.Orleans.Core.Abstractions
  - Microsoft.Orleans.CodeGenerator.MSBuild
  - Interfaces

Orleans Getting Started

HelloWorld

```
public class HelloWorld : Orleans.Grain, IHelloWorld
{
    private readonly ILogger _logger;

    public HelloWorld(ILogger<HelloWorld> logger) { _logger = logger; }

    public Task<string> SayHello(string name)
    {
        _logger.LogInformation("SayHello: name = '{name}', ", name);

        return Task.FromResult($"Hello, {name}, welcome to Orleans");
    }
```

Orleans Getting Started

Host

- Console
- requires
  - Microsoft.Orleans.Server
  - (Microsoft.Extensions.Hosting)
  - (Microsoft.Extensions.Logging.Console)
  - Interfaces
  - Grains

Orleans Getting Started

Program.cs

```
    var builder = new HostBuilder()
        .UseOrleans(c =>
        {
            c.UseLocalhostClustering()
            .Configure<ClusterOptions>(options =>
            {
                options.ClusterId = "dev";
                options.ServiceId = "HelloWorld";
            })
            .ConfigureApplicationParts(
                parts =>
parts.AddApplicationPart(typeof(HelloWorld).Assembly).WithReferences())
            .ConfigureLogging(logging => logging.AddConsole());
        });

    var host = builder.Build();
    await host.StartAsync();
```

Orleans Getting Started

Client

- Console
- requires
  - Microsoft.Orleans.Client
  - Interfaces

Orleans Getting Started

Client Setup

```
var client = new ClientBuilder()
    .UseLocalhostClustering()
    .Configure<ClusterOptions>(options =>
    {
        options.ClusterId = "dev";
        options.ServiceId = "HelloWorld";
    })
    .ConfigureLogging(logging => logging.AddConsole())
    .Build();

await client.Connect();
Console.WriteLine("Client successfully connected to silo host \n");
```

Orleans Getting Started

Client calling host

```
var friend = client.GetGrain<IHelloWorld>(0);
var response = await friend.SayHello("Good morning, HelloGrain!");
```

Implementing Orleans actors

ORLEANS BASICS

Orleans Basics

Grains

- must implement grain interface (C# interface)
- must extend `Orleans.Grain` base class
- optionally override `OnActivateAsync`/`OnDeactivateAsync`
  - for notification of activation/deactivation
  - exceptions will cancel activation
  - deactivation is not guaranteed (e.g., server crash)
- primary key available via `GetPrimaryKey`

Orleans Basics

Grain References

- grains are never accessed directly
- clients must obtain a "grain reference"
- pass identity to `GrainFactory.GetGrain`
- references can be used as typical .NET objects

Orleans Basics

Interfaces

- define the (strongly-tyoed) interaction between sender and recipient
- conceptually similar to older approaches (CORBA, DCOM, etc)
- ... but tweaked to reflect Orleans' particular opinions
- used to ensure implementation and client-proxy correctness
- basis for code generation during build

Orleans Basics

Interfaces

- must extend an "identity interface"
  - describes the kind of primary key
  - `IGrainWithGuidKey`
  - `IGrainWithIntegerKey`
  - `IGrainWithStringKey`
  - `IGrainWithGuidCompoundKey`
  - `IGrainWithIntegerCompoundKey`
- methods must return `Task`

More

Other topics

- Persistent grains
- Timers and reminders
- Streams
- Heterogenous Silos
- Evolving/versioning Grains
- Event sourcing
- Transactions

Summary

Microsoft Orleans

- virtual actor system
- built on top of CLR
- providing easy language-to-messaging syntax/semantics
- capable of supporting high scale/load

Where to go to get more

RESOURCES

Resources

Code/Github

- Orleans OSS repository
  - https://github.com/dotnet/orleans
- Official docs
  - https://learn.microsoft.com/en-us/dotnet/orleans/
- OrleansContrib
  - https://github.com/orgs/OrleansContrib
- Awesome-Orleans
  - https://github.com/OrleansContrib/Awesome-

Resources

Books

- "Introducing Microsoft Orleans"
  - https://www.amazon.com/Introducing-Microsoft-Orleans-Implementing-Cloud-Native/dp/148428013X
- "Microsoft Orleans for Developers"
  - https://www.amazon.com/Microsoft-Orleans-Developers-Cloud-Native-Distributed/dp/1484281667
- "Distributed .NET w/Microsoft Orleans"

Credentials

Who is this guy?

- Architect, Engineering Manager/Leader, "force multiplier"
  - http://www.newardassociates.com
  - http://blogs.newardassociates.com
- Sr Distinguished Engineer, Capital One
- Educative (http://educative.io) Author
  - *Performance Management for Engineering Managers*
- Books